



Especialización en Diseño y Desarrollo de Videojuegos

ACA 2

Asignatura: Diseño y construcción de ambientes interactivos en motores de videojuegos

Docente: Mateo Mancera Camacho

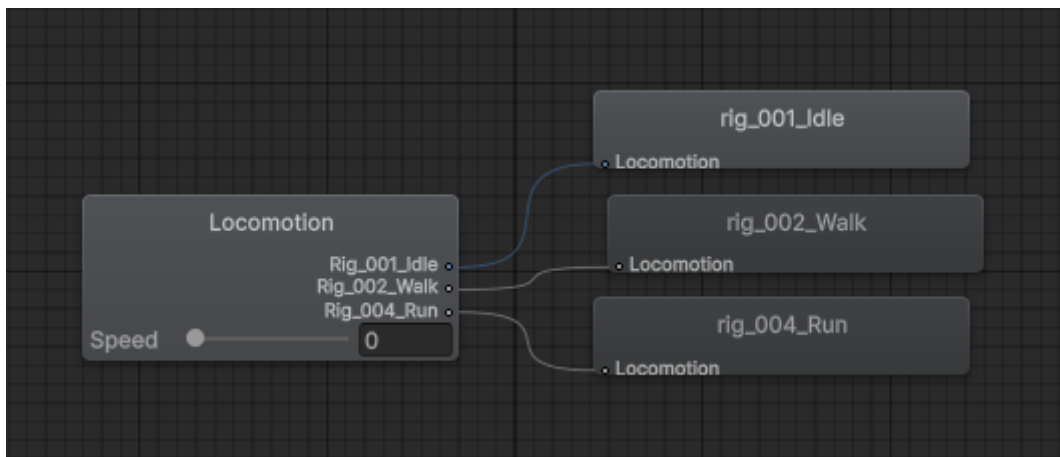
Presentado por:
Daniel Gómez Giraldo
Andrés Felipe Uribe Rodríguez
David Ernesto Rueda López

Cambios respecto al ACA 1

- Cambiamos el movimiento de la cámara para utilizar el delta de la posición del mouse en vez del teclado
- Añadimos un personaje animado y texturizado
- Extendimos los estados de la FSM del personaje para incluir:
 - Estado de Jump_End para manejar la animación final del salto
 - Estado de Aim para poder apuntar con un arma y disparar, este estado además produce un pequeño line renderer para ayudar a apuntar
- Implementamos texturas para ayudar en la visibilidad del escenario blackout
- Construimos un 3er nivel enfocado en combate, y una pequeña escena sencilla para marcar el final del nivel
- Implementamos transiciones y animaciones para separar los niveles y poder moverse entre uno y otro sin obstaculizar la cámara
- Añadimos lógica para que los cubos de los puzzles puedan respawnear por cuenta propia si caen al vacío
- Creamos enemigos con su propia FSM e interacción con navmesh para detectar, perseguir, y atacar al personaje
- El personaje principal ahora tiene un sistema de HP que lo respawnea al llegar a 0

Animación con Blend Trees

Usamos blend trees para manejar la locomoción del personaje principal. El nodo de blend tree toma las tres animaciones principales de movimiento (Idle, Walk, Run) y las combina con un fade de una a otra dependiendo del valor de "Speed". El valor de Speed es calculado con la magnitud de la acción de movimiento en el Input system, para luego poder usada entre 0 a 1 con los análogos de un control, o manualmente con una tecla, para que el personaje tenga diferentes velocidades de movimiento y las animaciones correspondientes.



Inteligencia artificial

Para la inteligencia artificial del nivel se usó el componente de NavMesh de Unity para definir las superficies que los enemigos podían o no cruzar.

Dentro de la configuración de la superficie se excluyó al jugador, los enemigos y el pickup del arma dentro del bake para no generar gaps innecesarios. Esto mediante asignar layers que pudieran ser ignoradas por el manager del NavMesh. A las cajas que el jugador puede empujar se les asignó el componente de NavMesh Obstacle con un carve permanente para

que pudieran modificar la superficie a medida que el jugador las empujara. A las puertas y paredes destructibles se les puso también un NavMesh Obstacle, esta vez con la opción de “Carve Only Stationary”, pues son objetos que no van a estar en constante movimiento como las cajas.

Para los enemigos se usaron NavMesh Agents con la siguiente máquina de estados.

- Patrol (patrulla): El enemigo se mueve entre los waypoints definidos en el mapa, utilizando la superficie de NavMesh para calcular su recorrido. Si el jugador entra dentro de su rango de persecución, cambia al estado Pursuit.
- Pursuit (persecución): El enemigo cambia su destino al jugador y se mueve hacia él, siempre y cuando este se mantenga dentro de su rango de persecución. Si el jugador se acerca a su rango de ataque, cambia al estado Attack. Si el jugador se aleja y sale de su rango de persecución, vuelve al estado Patrol.
- Attack: El enemigo empieza a disparar al jugador mientras que esté dentro de su rango. Durante este estado el enemigo está estático siempre que el jugador no se acerque al borde de su rango de ataque, donde lo volverá a perseguir sin dejar de disparar si este sigue en su línea de fuego. Si el jugador sale del rango de ataque, pasa al estado Pursuit.

Para el ataque, el enemigo tiene una función que cuando está en estado Attack modifica su rotación para que mire hacia el jugador. Los disparos de los enemigos usan una corrutina que hace que el enemigo dispare cada 1 segundo cuando está en estado Attack, y para determinar cuándo debe disparar se implementó un raycast desde el cañón del enemigo que detecta si el jugador está en frente para disparar

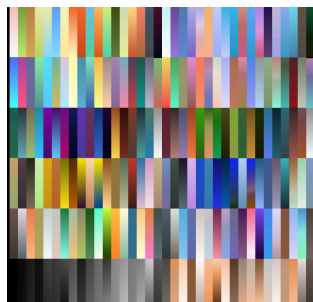
Interfaz de usuario

El HUD del jugador contiene una barra que refleja la vida del jugador, situada en la parte superior izquierda de la pantalla. Esta barra está conectada a la salud del jugador para que se actualice constantemente si el jugador recibe daño de los enemigos.

Además, se implementó una barra de vida encima de los enemigos en modo world para que se mueva junto con ellos. Desde el script de la IA se maneja la rotación del canvas para que esté siempre frente a la cámara.

Optimización

Una de las características de este proyecto es que desde el principio utilizamos una textura de atlas para tener un sólo material que pudiera dar color a diferentes objetos al tiempo. Esta textura nos dio el beneficio de tener un número reducido de draw calls desde el inicio sin necesidad de tener que reducir el número de texturas usadas posteriormente.



Además, los niveles están diseñados como cuartos pequeños y organizados de manera vertical en la escena lo que reduce considerablemente la cantidad de polígonos que se necesitan cargar frente a la cámara en un momento específico.



Un pico considerable a optimizar en el futuro es el uso de físicas en determinados momentos de la experiencia, sin embargo como estudiantes aún no tenemos las herramientas para optimizar este aspecto, y la información en línea como tutoriales al respecto se sale de nuestro conocimiento actual.

